

Top 30 LeetCode SQL Questions and Answers– Part 3

by tarun reddy

1. Longest Consecutive Login Streak per User

Problem

Find the longest streak of consecutive login days for each user.

Input Table: Logins

user_id	login_date
1	2024-01-01
1	2024-01-02
1	2024-01-04
2	2024-01-01

Expected Output

user_id	longest_streak
1	2
2	1

```
WITH cte AS (  
  SELECT user_id,  
         login_date,  
         ROW_NUMBER() OVER (  
           PARTITION BY user_id ORDER BY login_date  
         ) AS rn  
  FROM Logins  
)  
grp AS (  
  SELECT user_id,  
         DATE_SUB(login_date, INTERVAL rn DAY) AS grp_key  
  FROM cte  
)  
SELECT user_id,  
       COUNT(*) AS longest_streak  
FROM grp  
GROUP BY user_id, grp_key;
```

2. Rolling 7-Day Average of Orders

Problem

Calculate the rolling 7-day average of daily orders.

Input Table: Orders

order_date	order_id
2024-01-01	1
2024-01-03	2
2024-01-07	3

Expected Output

order_date	avg_7day
2024-01-01	1.00
2024-01-03	1.00
2024-01-07	0.43

```
WITH daily AS (  
  SELECT order_date, COUNT(*) AS orders  
  FROM Orders  
  GROUP BY order_date  
)  
SELECT d1.order_date,  
  ROUND(  
    (SELECT AVG(d2.orders)  
     FROM daily d2  
     WHERE d2.order_date BETWEEN DATE_SUB(d1.order_date, INTERVAL 6 DAY)  
           AND d1.order_date), 2) AS avg_7day  
FROM daily d1;
```

3. Day-7 Retention Rate (Cohort Analysis)

Problem

For each signup date, calculate the percentage of users who return after 7 days.

Input Table: Users

user_id	signup_date
1	2024-01-01
2	2024-01-01
3	2024-01-02

Input Table: Activity

user_id	activity_date
1	2024-01-08

Expected Output

signup_date	retention_rate
2024-01-01	0.50
2024-01-02	0.00

```
SELECT u.signup_date,  
  ROUND(  
    SUM(  
      CASE WHEN a.activity_date =  
        DATE_ADD(u.signup_date, INTERVAL 7 DAY)  
        THEN 1 ELSE 0 END  
    ) / COUNT(*), 2) AS retention_rate
```

```
FROM Users u
LEFT JOIN Activity a
  ON u.user_id = a.user_id
GROUP BY u.signup_date;
```

4. Funnel Conversion Rate

Problem

Calculate session → purchase conversion rate.

Input Table: Events

user_id	event_type
1	session
1	purchase
2	session

Expected Output

conversion_rate
0.50

```
SELECT ROUND(
  SUM(event_type = 'purchase') /
  SUM(event_type = 'session'), 2
) AS conversion_rate
FROM Events;
```

5. Month-over-Month Revenue Growth

Problem

Calculate month-over-month revenue growth percentage.

Input Table: Orders

order_date	amount
2024-01-15	1000
2024-02-10	1200

Expected Output

month	mom_growth
2024-02	20.00

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
         SUM(amount) AS revenue
  FROM Orders
  GROUP BY month
```

```

)
SELECT month,
       ROUND(
         (revenue - LAG(revenue) OVER (ORDER BY month)) /
         LAG(revenue) OVER (ORDER BY month) * 100, 2
       ) AS mom_growth
FROM monthly
WHERE LAG(revenue) OVER (ORDER BY month) IS NOT NULL;

```

6. Users Who Churned (Inactive for 30 Days)

Problem

Identify users whose last activity was more than 30 days ago.

Input Table: Activity

user_id	activity_date
1	2024-01-01
2	2024-02-15
3	2024-01-10

(Reference date: 2024-02-20)

Expected Output

user_id
1
3

```

SELECT user_id
FROM Activity
GROUP BY user_id
HAVING MAX(activity_date) <
       DATE_SUB('2024-02-20', INTERVAL 30 DAY);

```

7. Customers With Increasing Spend (3 Consecutive Months)

Problem

Find customers whose monthly spend strictly increases for three consecutive months.

Input Table: Orders

customer_id	order_date	amount
1	2024-01-10	100
1	2024-02-10	150
1	2024-03-10	200

Expected Output

customer_id

1

```
WITH monthly AS (  
  SELECT customer_id,  
         DATE_FORMAT(order_date,'%Y-%m') AS ym,  
         SUM(amount) AS spend  
  FROM Orders  
  GROUP BY customer_id, ym  
)  
SELECT DISTINCT m1.customer_id  
FROM monthly m1  
JOIN monthly m2  
  ON m1.customer_id = m2.customer_id  
 AND DATE_ADD(m1.ym, INTERVAL 1 MONTH) = m2.ym  
JOIN monthly m3  
  ON m2.customer_id = m3.customer_id  
 AND DATE_ADD(m2.ym, INTERVAL 1 MONTH) = m3.ym  
WHERE m1.spend < m2.spend  
      AND m2.spend < m3.spend;
```

8. Daily Cumulative Unique Users

Problem

Calculate cumulative distinct users up to each day.

Input Table: Logins

login_date	user_id
2024-01-01	1
2024-01-02	2
2024-01-03	3

Expected Output

login_date	cumulative_users
2024-01-01	1
2024-01-02	2
2024-01-03	3

```
SELECT d.login_date,  
       COUNT(DISTINCT l.user_id) AS cumulative_users  
FROM Logins d  
JOIN Logins l  
  ON l.login_date <= d.login_date  
GROUP BY d.login_date;
```

9. Orders Above Daily Average

Problem

Find orders whose amount is higher than the daily average.

Input Table: Orders

order_id	order_date	amount
1	2024-01-01	100
2	2024-01-01	200
3	2024-01-02	50
4	2024-01-02	150

Expected Output

order_id	amount
2	200
4	150

```
SELECT order_id, amount
FROM (
  SELECT order_id,
         amount,
         AVG(amount) OVER (
           PARTITION BY order_date
           ) AS avg_amt
  FROM Orders
) t
WHERE amount > avg_amt;
```

10. Peak Order Hour per Day

Problem

Find the hour with maximum orders for each day.

Input Table: Orders

order_time
2024-01-01 10:15
2024-01-01 10:45
2024-01-01 14:30
2024-01-02 09:10

Expected Output

order_date	hour	order_count
2024-01-01	10	2
2024-01-02	9	1

```

SELECT order_date, hour, order_count
FROM (
  SELECT DATE(order_time) AS order_date,
    HOUR(order_time) AS hour,
    COUNT(*) AS order_count,
    ROW_NUMBER() OVER (
      PARTITION BY DATE(order_time)
      ORDER BY COUNT(*) DESC
    ) AS rn
  FROM Orders
  GROUP BY DATE(order_time), HOUR(order_time)
) t
WHERE rn = 1;

```

11. First Purchase Date per Customer

Problem

Find the first purchase date for each customer.

Input Table: Orders

customer_id	order_date
1	2024-01-05
1	2024-01-20
2	2024-02-02

Expected Output

customer_id	first_order_date
1	2024-01-05
2	2024-02-02

```

SELECT customer_id,
  MIN(order_date) AS first_order_date
FROM Orders
GROUP BY customer_id;

```

12. Customers Who Purchased All Products

Problem

Find customers who purchased every available product at least once.

Input Table: Orders

customer_id	product_id
1	101

customer_id product_id

1 102

1 103

2 101

Input Table: Products

product_id

101

102

103

Expected Output

customer_id

1

```
SELECT customer_id
FROM Orders
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) =
(SELECT COUNT(*) FROM Products);
```

13. Top 2 Products by Revenue per Month

Problem

For each month, return the top 2 products by revenue.

Input Table: Orders

product_id	order_date	amount
------------	------------	--------

101	2024-01-05	300
-----	------------	-----

102	2024-01-10	500
-----	------------	-----

103	2024-01-12	200
-----	------------	-----

Expected Output

month	product_id	revenue
-------	------------	---------

2024-01	102	500
---------	-----	-----

2024-01	101	300
---------	-----	-----

```
WITH monthly AS (
  SELECT product_id,
    DATE_FORMAT(order_date,'%Y-%m') AS month,
    SUM(amount) AS revenue
  FROM Orders
  GROUP BY product_id, month)
```

```

)
SELECT month, product_id, revenue
FROM (
  SELECT *,
    DENSE_RANK() OVER (
      PARTITION BY month
      ORDER BY revenue DESC
    ) AS rk
  FROM monthly
) t
WHERE rk <= 2;

```

14. Users Logged In on Consecutive Days

Problem

Find users who logged in on at least two consecutive days.

Input Table: Logins

user_id	login_date
1	2024-01-01
1	2024-01-02
2	2024-01-01
2	2024-01-03

Expected Output

user_id
1

```

SELECT DISTINCT l1.user_id
FROM Logins l1
JOIN Logins l2
  ON l1.user_id = l2.user_id
 AND DATEDIFF(l2.login_date, l1.login_date) = 1;

```

15. Revenue Contribution Percentage by Product

Problem

Calculate each product's contribution percentage to total revenue.

Input Table: Orders

product_id	amount
101	300
102	700

Expected Output

product_id	contribution_pct
101	30.00
102	70.00

```
SELECT product_id,  
       ROUND(  
         SUM(amount) * 100.0 /  
         SUM(SUM(amount)) OVER (), 2  
       ) AS contribution_pct  
FROM Orders  
GROUP BY product_id;
```

16. Users Who Did Action A but Never Action B (Anti-Join)

Problem

Find users who logged in but never made a purchase.

Input Table: Events

user_id	event_type
1	login
1	purchase
2	login
3	login

Expected Output

user_id
2
3

```
SELECT DISTINCT user_id  
FROM Events  
WHERE event_type = 'login'  
AND user_id NOT IN (  
  SELECT user_id  
  FROM Events  
  WHERE event_type = 'purchase'  
);
```

17. Top Customer by Lifetime Value (LTV)

Problem

Find the customer with the highest total spend.

Input Table: Orders

customer_id	amount
-------------	--------

customer_id	amount
1	500
2	800
1	300

Expected Output

customer_id	total_spend
2	800

```
SELECT customer_id,  
       SUM(amount) AS total_spend  
FROM Orders  
GROUP BY customer_id  
ORDER BY total_spend DESC  
LIMIT 1;
```

18. Orders With Missing Delivery Records

Problem

Find orders that do not have a matching delivery record.

Input Table: Orders

order_id
1
2
3

Input Table: Deliveries

order_id
1
2

Expected Output

order_id
3

```
SELECT o.order_id  
FROM Orders o  
LEFT JOIN Deliveries d  
  ON o.order_id = d.order_id  
WHERE d.order_id IS NULL;
```

19. RFM – Recency per Customer

Problem

Calculate recency (days since last order) for each customer.
(Reference date: 2024-02-20)

Input Table: Orders

customer_id	order_date
1	2024-02-15
2	2024-01-01

Expected Output

customer_id	recency_days
1	5
2	50

```
SELECT customer_id,
       DATEDIFF('2024-02-20', MAX(order_date)) AS recency_days
FROM Orders
GROUP BY customer_id;
```

20. Identify Duplicate Transactions

Problem

Detect duplicate transactions with same user, date, and amount.

Input Table: Transactions

user_id	trans_date	amount
1	2024-01-01	200
1	2024-01-01	200
2	2024-01-02	150

Expected Output

user_id	trans_date	amount
1	2024-01-01	200

```
SELECT user_id, trans_date, amount
FROM Transactions
GROUP BY user_id, trans_date, amount
HAVING COUNT(*) > 1;
```

21. Orders Above Customer's Average Order Value

Problem

Find orders whose amount is above the customer's average.

Input Table: Orders

order_id	customer_id	amount
1	1	100
2	1	300
3	2	50

Expected Output

order_id	customer_id	amount
2	1	300

```
SELECT order_id, customer_id, amount
FROM (
  SELECT order_id, customer_id, amount,
         AVG(amount) OVER (PARTITION BY customer_id) AS avg_amt
  FROM Orders
) t
WHERE amount > avg_amt;
```

22. Customers With No Orders in the Last 60 Days

Problem

Find customers whose latest order was more than 60 days ago.

(Reference date: 2024-02-20)

Input Table: Orders

customer_id	order_date
1	2023-11-01
2	2024-02-01
3	2023-10-01

Expected Output

customer_id
1
3

```
SELECT customer_id
FROM Orders
GROUP BY customer_id
HAVING MAX(order_date) <
  DATE_SUB('2024-02-20', INTERVAL 60 DAY);
```

23. Rank Products by Sales Volume Within Category

Problem

Rank products by units sold within each category.

Input Table: Products

product_id	category
101	Electronics
102	Electronics
201	Home

Input Table: Orders

product_id	quantity
101	50
102	80
201	20

Expected Output

category	product_id	rank
Electronics	102	1
Electronics	101	2
Home	201	1

```
SELECT category, product_id,
       DENSE_RANK() OVER (
         PARTITION BY category
         ORDER BY units DESC
       ) AS rank
FROM (
  SELECT p.category, o.product_id,
         SUM(o.quantity) AS units
  FROM Orders o
  JOIN Products p
    ON o.product_id = p.product_id
  GROUP BY p.category, o.product_id
) t;
```

24. Peak Revenue Day

Problem

Find the date with the highest revenue.

Input Table: Orders

order_date	amount
------------	--------

2024-01-01	1000
------------	------

2024-01-02	1500
------------	------

2024-01-03	1200
------------	------

Expected Output

order_date	revenue
2024-01-02	1500

```
SELECT order_date,  
       SUM(amount) AS revenue  
FROM Orders  
GROUP BY order_date  
ORDER BY revenue DESC  
LIMIT 1;
```

25. Running Total of Revenue per Day

Problem

Calculate the running cumulative revenue by date.

Input Table: Orders

order_date	amount
2024-01-01	100
2024-01-02	200
2024-01-03	150

Expected Output

order_date	running_revenue
2024-01-01	100
2024-01-02	300
2024-01-03	450

```
SELECT order_date,  
       SUM(amount) OVER (ORDER BY order_date) AS running_revenue  
FROM Orders;
```

26. Customers With More Than 3 Orders in Any Month

Problem

Find customers who placed more than 3 orders in the same month.

Input Table: Orders

customer_id	order_date
1	2024-01-01
1	2024-01-05
1	2024-01-10
1	2024-01-15

Expected Output

customer_id	month
1	2024-01

```
SELECT customer_id,  
       DATE_FORMAT(order_date,'%Y-%m') AS month  
FROM Orders  
GROUP BY customer_id, month  
HAVING COUNT(*) > 3;
```

27. Products Never Sold

Problem

Find products that never appeared in orders.

Input Table: Products

product_id
101
102
103

Input Table: Orders

product_id
101
102

Expected Output

product_id
103

```
SELECT p.product_id  
FROM Products p  
LEFT JOIN Orders o  
  ON p.product_id = o.product_id  
WHERE o.product_id IS NULL;
```

28. Customers With Increasing Spend Over Time

Problem

Find customers whose spend increases with each purchase.

Input Table: Orders

customer_id	order_date	amount
1	2024-01-01	100
1	2024-02-01	150
1	2024-03-01	200

Expected Output

customer_id
1

```
SELECT DISTINCT customer_id
FROM (
  SELECT customer_id, amount,
    LAG(amount) OVER (
      PARTITION BY customer_id
      ORDER BY order_date
    ) AS prev_amt
  FROM Orders
) t
WHERE amount > prev_amt;
```

29. Swap Values Using CASE

Problem

Swap 'Manager' and 'Lead' values in a single update.

Input Table: Employees

id	role
1	Manager
2	Lead
3	Engineer

Expected Output

id	role
1	Lead
2	Manager
3	Engineer

```
UPDATE Employees
SET role = CASE
  WHEN role = 'Manager' THEN 'Lead'
  WHEN role = 'Lead' THEN 'Manager'
  ELSE role
```

END;

30. Nth Highest Salary (2nd Highest)

Problem

Find the 2nd highest salary.

Input Table: Employee

id	salary
1	3000
2	2000
3	1000

Expected Output

salary
2000

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```